CS 301
High-Performance Computing

Lab-04

Particle Interpolation and Mover Performance Analysis

Tanishkumar Patel (202301411)
Neelabh Rana (202301476)

Contents

1 Introduction **3**

2 Hardware Details **3**
2.1 Local System (Lab 207) 3
2.2 HPC Cluster 3

3 Experiment 01: Scaling with Number of Particles **3**
3.1 Case 1: $N_x = 250, N_y = 100$ 4
3.2 Case 2: $N_x = 500, N_y = 200$ 6
3.3 Case 3: $N_x = 1000, N_y = 400$ 8

4 Experiment 02: Consistency of Interpolation Time **9**

5 Experiment 03: Mover Operation and OpenMP Parallelization **11**
5.1 Timing Breakdown (Triple Curve Plots) 11
5.2 Mover Serial vs. Parallel Execution 13
5.3 Parallel Speedup Analysis 13

6 Overall Approach and Analysis **15**
6.1 Implementation Schematic 15
6.2 Performance vs. Problem Index 15
6.3 Architectural Commentary 16
6.4 Speedup Calculation 17
6.5 Discussion: Parallelizing Interpolation 17

1 Introduction

This report analyzes the performance of a scattered point-to-mesh interpolation scheme and a particle tracking phase known as the "Mover." We evaluate computational scaling across three grid configurations ($Nx \times Ny$), execution consistency over multiple iterations, and the efficiency gains achieved through OpenMP parallelization.

2 Hardware Details

2.1 Local System (Lab 207)

- **Processor:** Intel Core i5-12500
- **Architecture:** 6 Cores / 12 Threads
- **Cache:** L1: 288K, L2: 7.5M, L3: 18M

2.2 HPC Cluster

- **Processor:** Intel Xeon E5-2640 v3
- **Architecture:** 2 Sockets, 8 Cores per socket (16 Cores total)
- **Cache:** L3: 20MB

3 Experiment 01: Scaling with Number of Particles

We measured execution time for particle counts $\{10^2, 10^4, 10^6, 10^8, 10^9\}$ for three grid cases.

3.1 Case 1: $N_x = 250, N_y = 100$

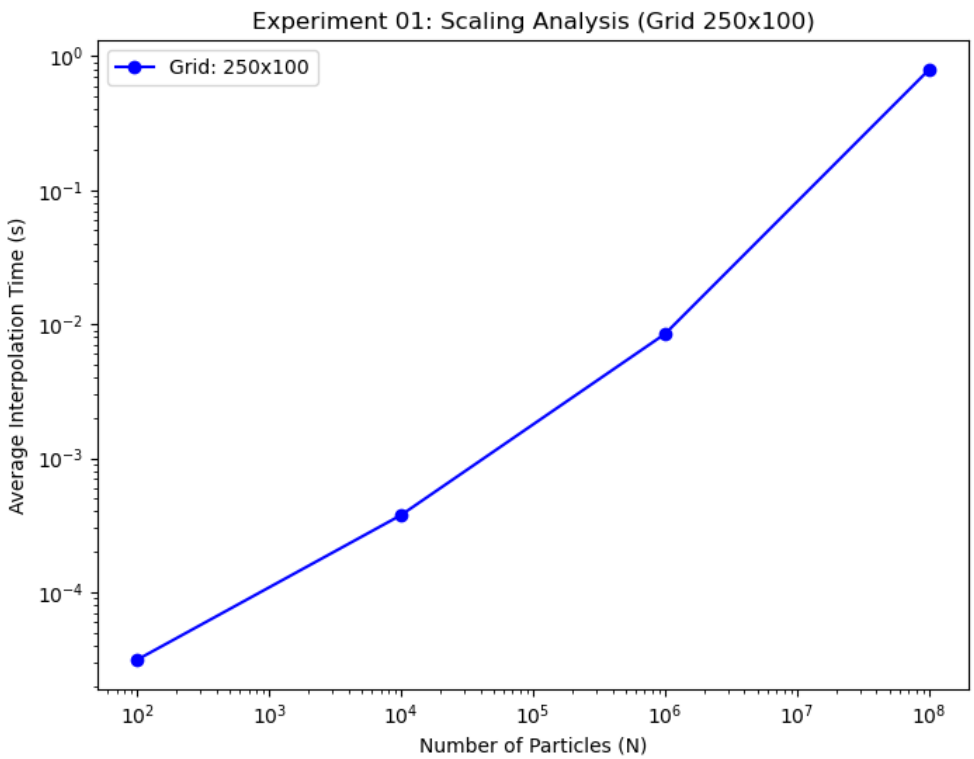

Figure 1: Scaling Analysis (Local PC): Grid 250×100

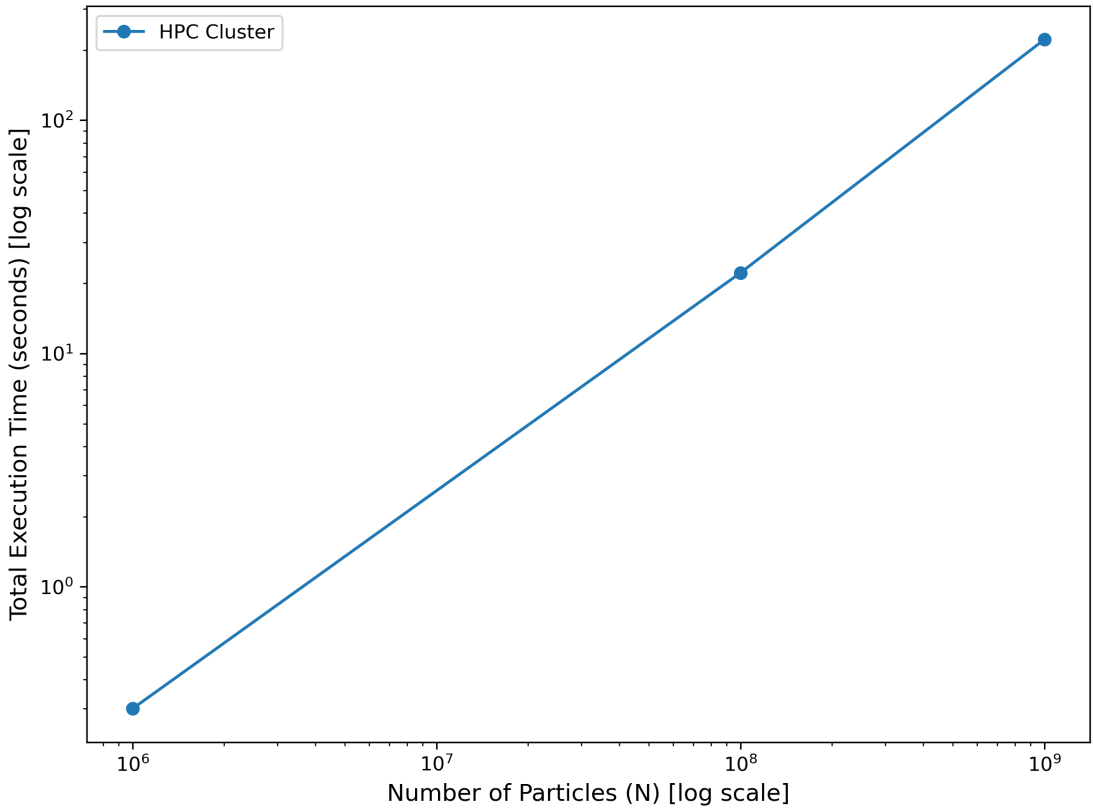

Figure 2: Scaling Analysis (HPC Cluster): Grid 250×100

3.2 Case 2: $N_x = 500, N_y = 200$

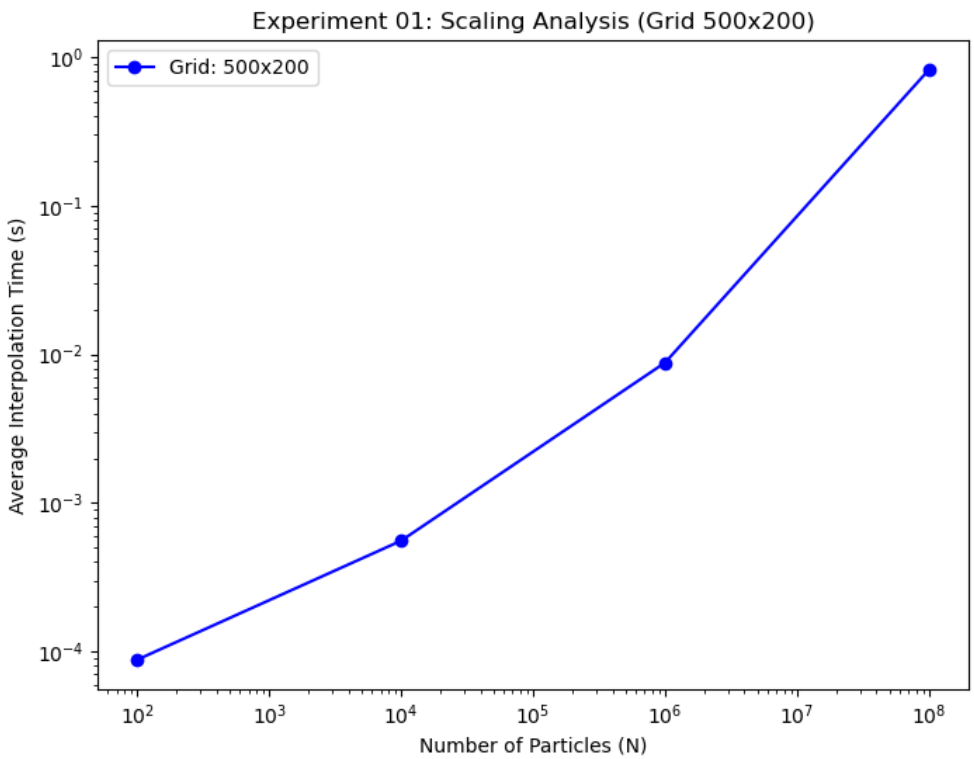

Figure 3: Scaling Analysis (Local PC): Grid 500×200

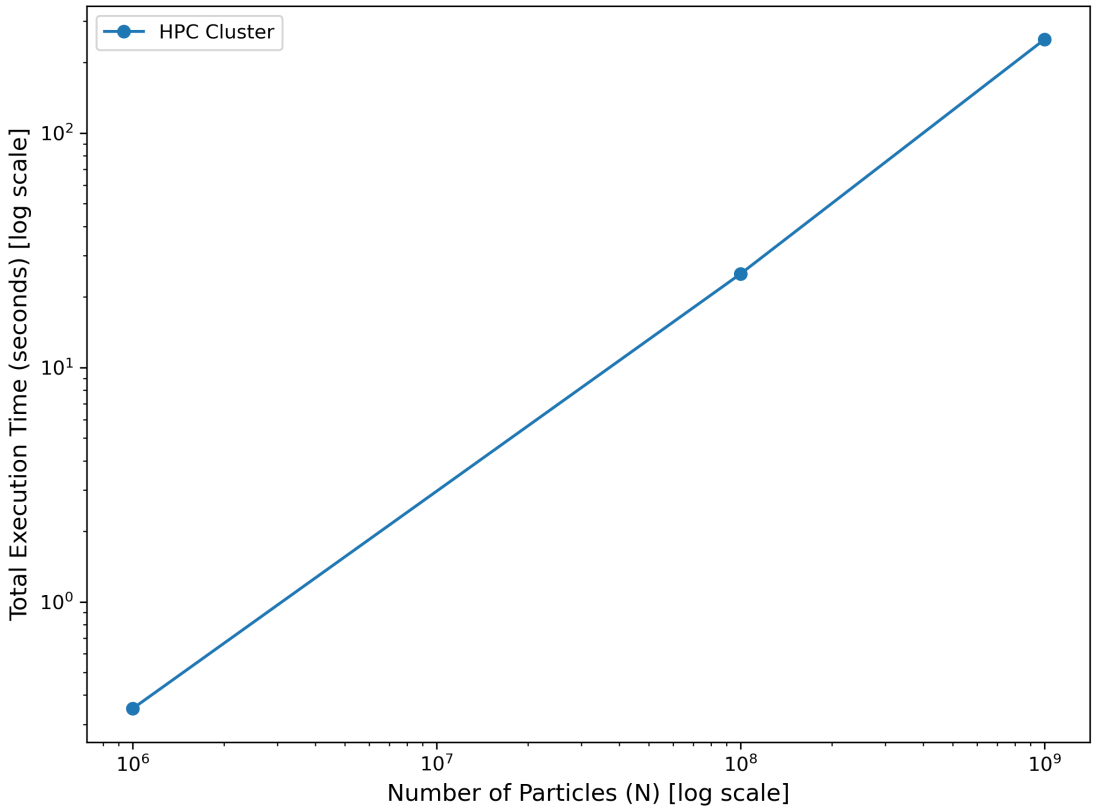

Figure 4: Scaling Analysis (HPC Cluster): Grid 500×200

3.3 Case 3: $N_x = 1000, N_y = 400$

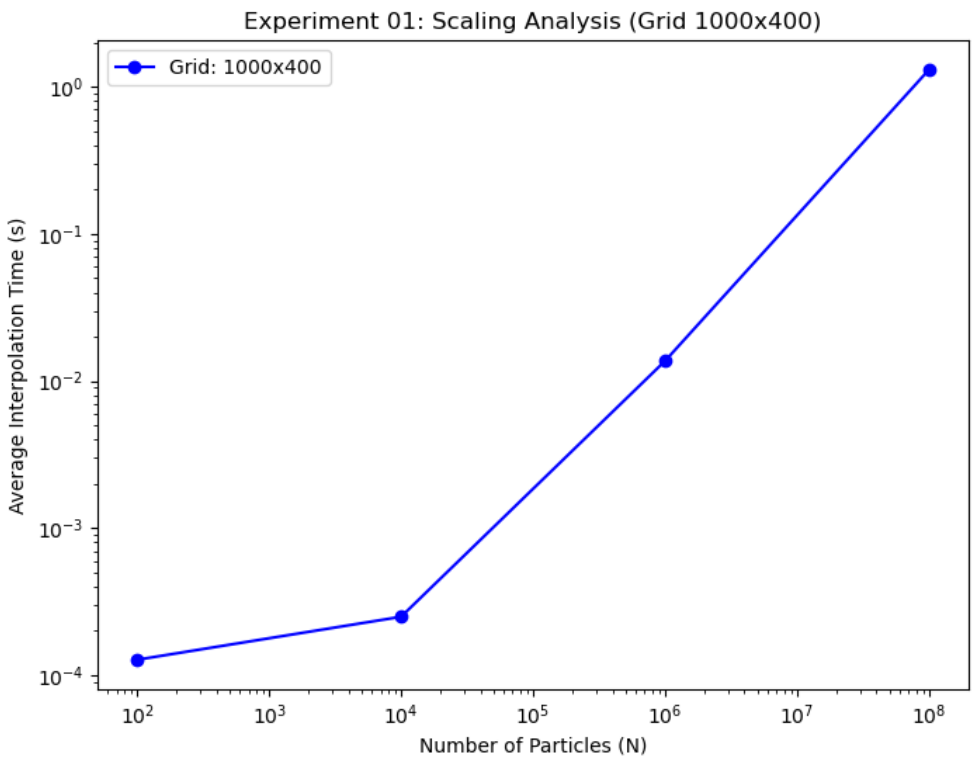

Figure 5: Scaling Analysis (Local PC): Grid 1000×400

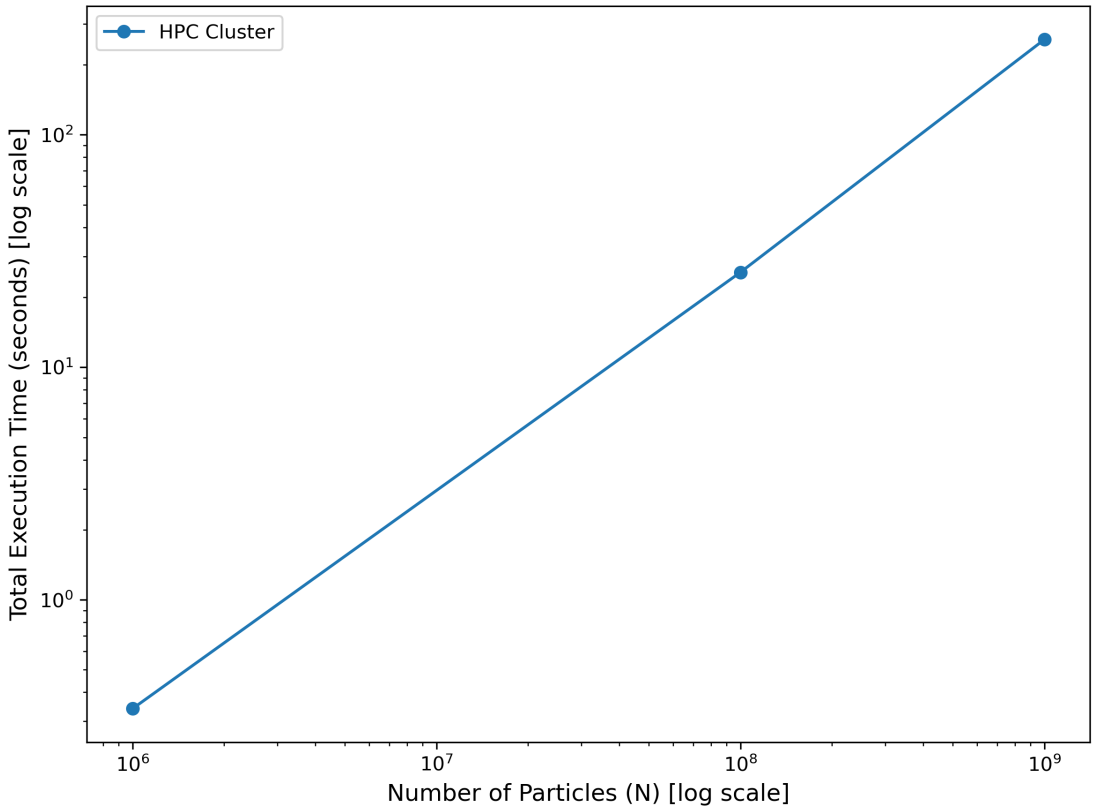

Figure 6: Scaling Analysis (HPC Cluster): Grid 1000×400

4 Experiment 02: Consistency of Interpolation Time

We evaluate the stability of the interpolation algorithm by measuring the execution time across 10 consecutive iterations using a fixed workload of 10^8 particles.

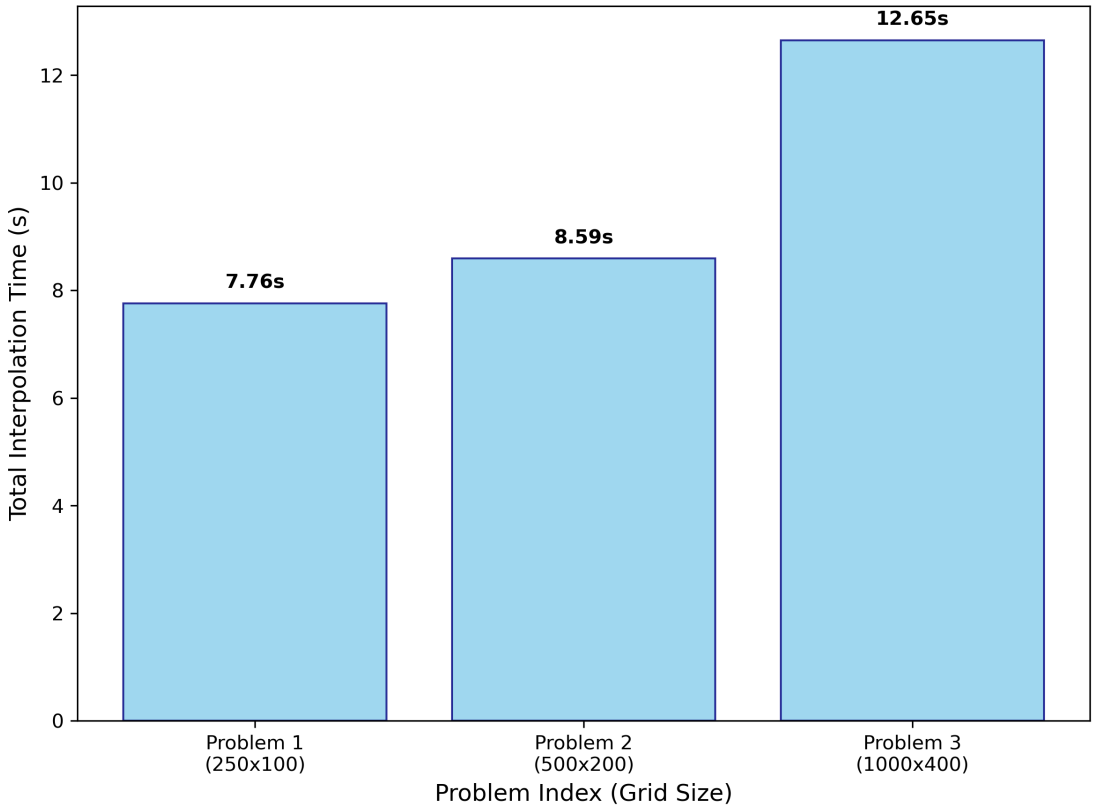

Figure 7: Interpolation Time Consistency: Local PC (10^8 particles)

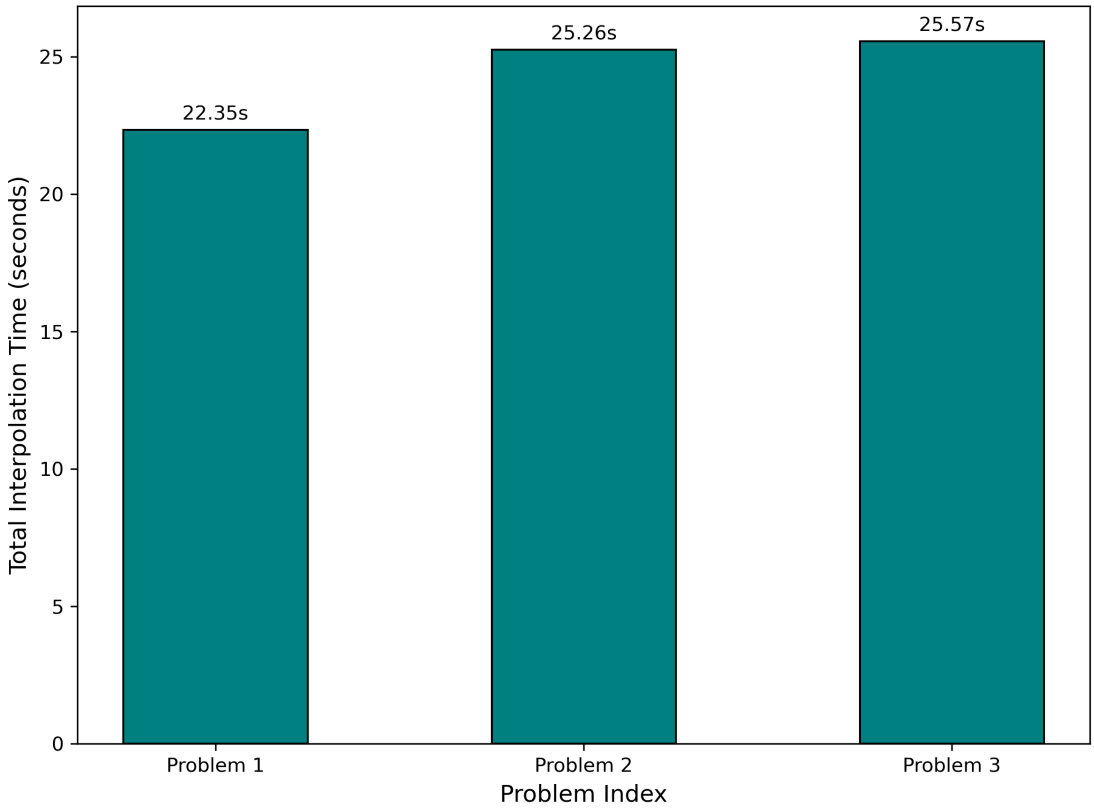

Figure 8: Interpolation Time Consistency: HPC Cluster (10^8 particles)

5 Experiment 03: Mover Operation and OpenMP Parallelization

5.1 Timing Breakdown (Triple Curve Plots)

The following plots illustrate the relationship between **Interpolation Time**, **Mover Time**, and **Total Execution Time** per iteration.

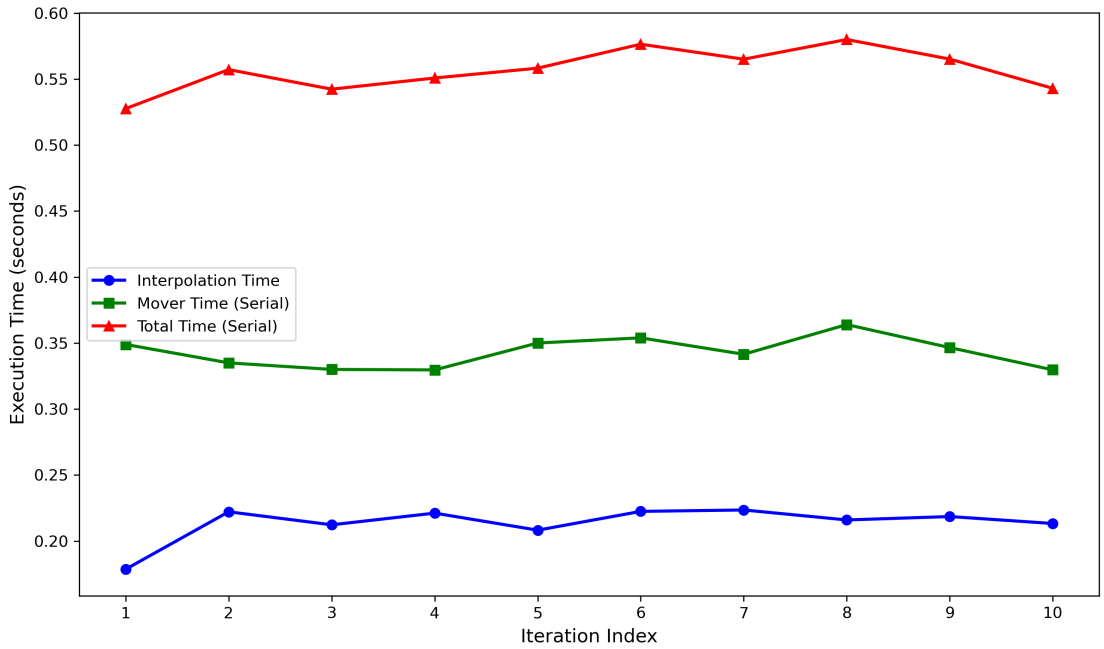

Figure 9: Component Timing per Iteration: Local PC

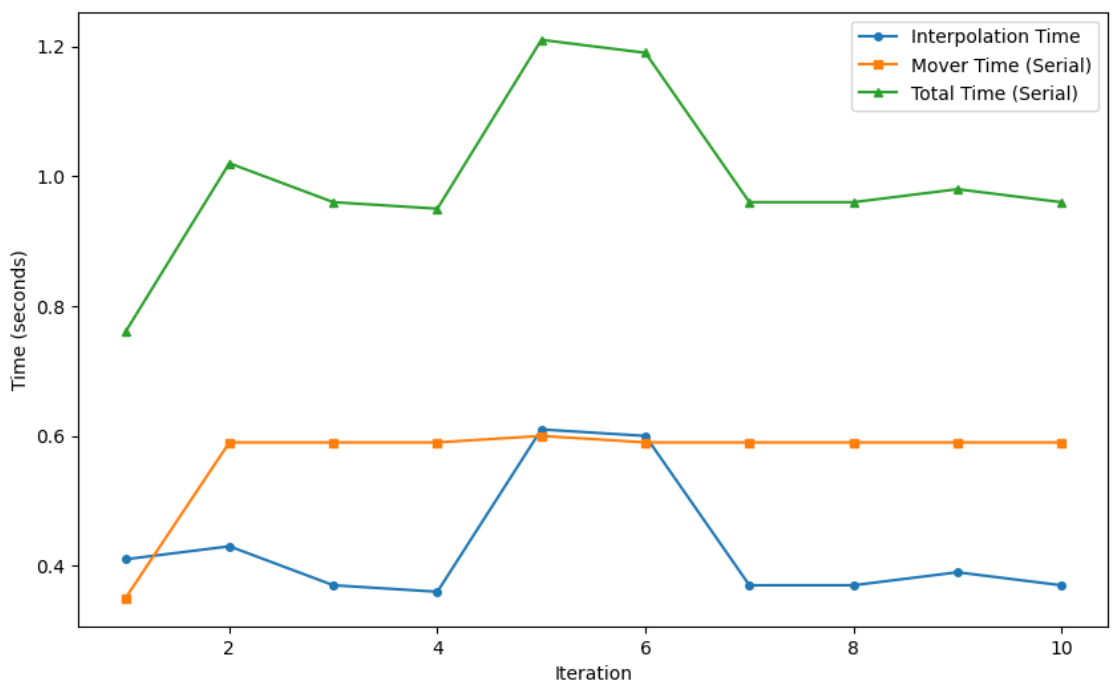

Figure 10: Component Timing per Iteration: HPC Cluster

5.2 Mover Serial vs. Parallel Execution

Comparison of the Mover execution time for serial implementation against the OpenMP parallel implementation. This visualizes the raw reduction in compute time.

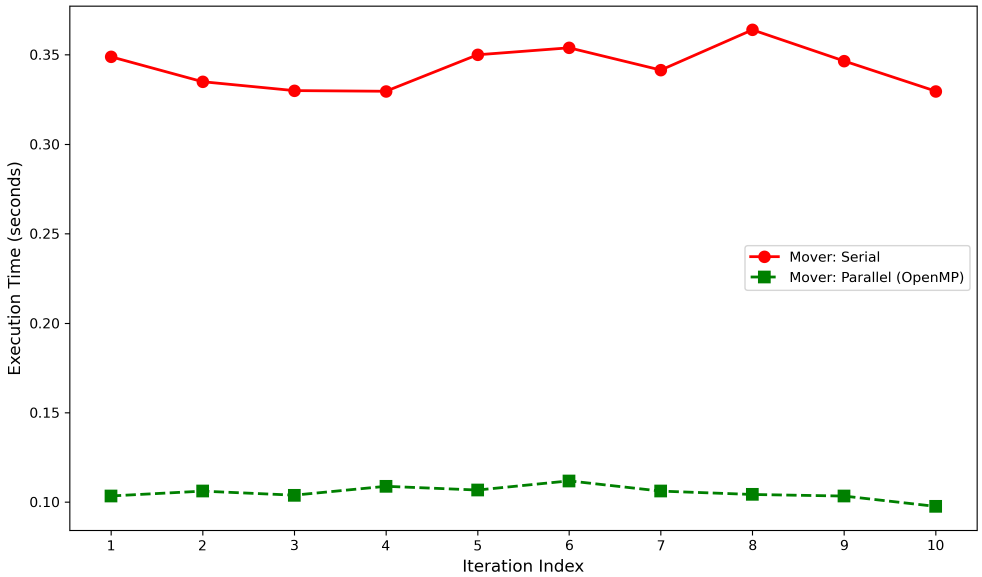

Figure 11: Mover Execution Time: Serial vs. Parallel Implementation

5.3 Parallel Speedup Analysis

Speedup (S) is computed as $S = \frac{T_{serial}}{T_{parallel}}$. The following plot displays the efficiency gain across the hardware threads.

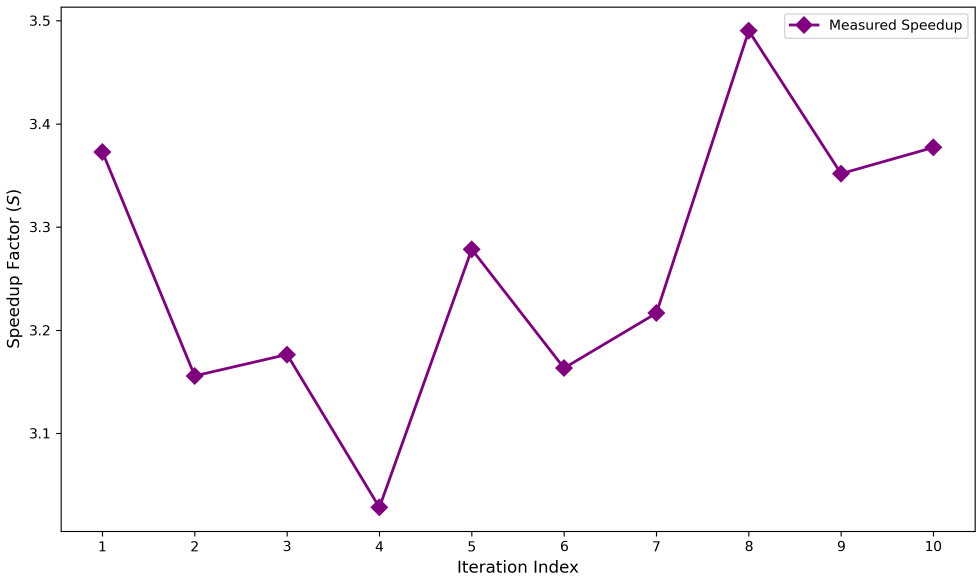

Figure 12: Speedup Achieved by OpenMP Parallelization of the Mover Operation

6 Overall Approach and Analysis

6.1 Implementation Schematic

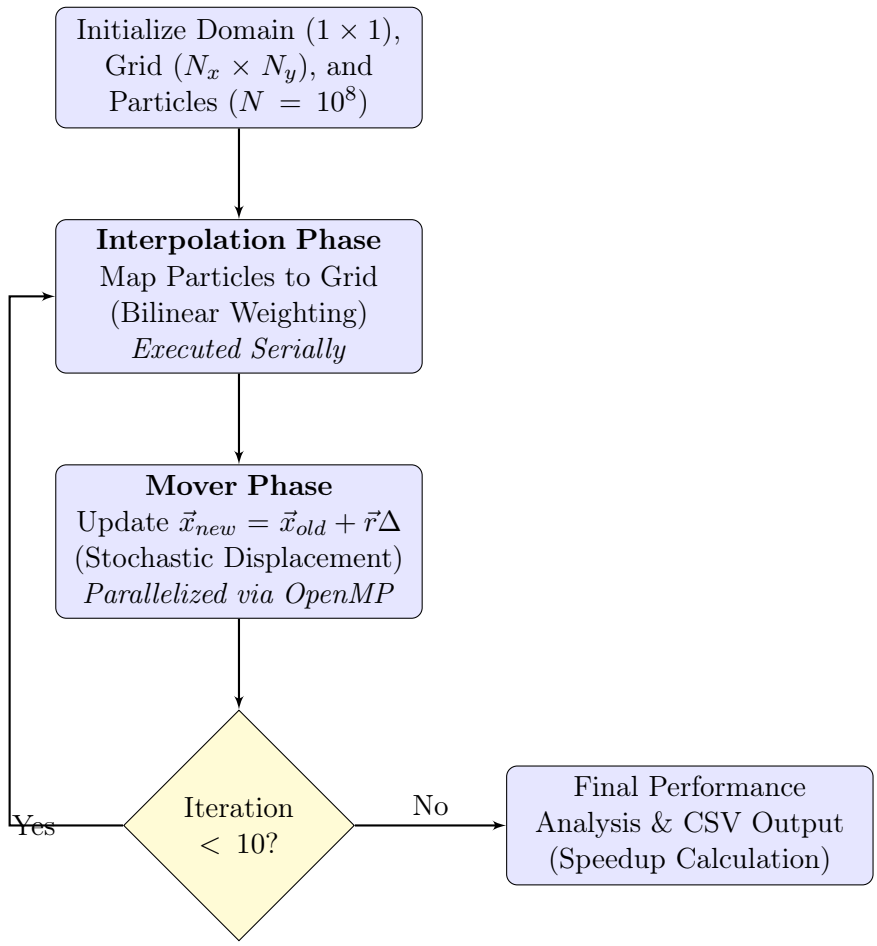

Figure 13: Schematic Logic Flow: Serial Interpolation vs. Parallelized Stochastic Mover.

6.2 Performance vs. Problem Index

Below is the bar plot comparing execution times for different problem indices across both systems.

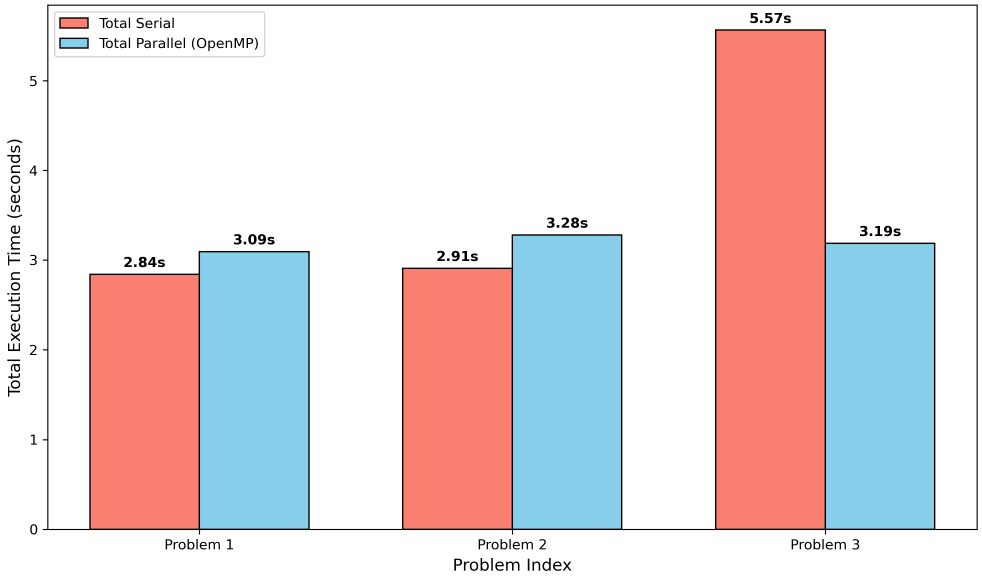

Figure 14: Execution Time vs. Problem Index

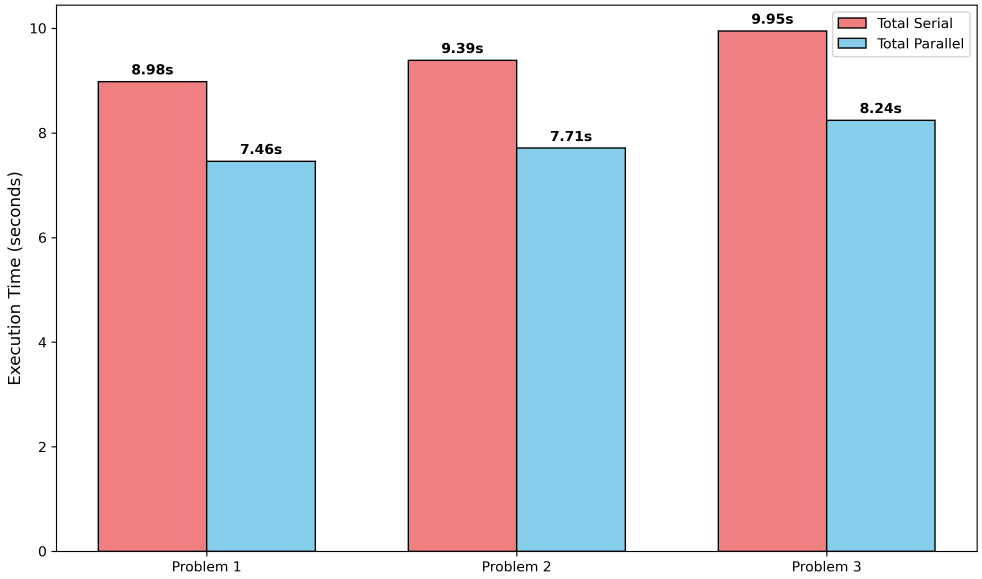

Figure 15: Execution Time vs. Problem Index

6.3 Architectural Commentary

The observed performance differences between the Lab PC and the HPC Cluster are primarily due to:

- **Cache Hierarchy:** The Xeon processor's larger L3 cache (20MB) handles larger particle counts (10^9) more efficiently than the i5-12500.

- **Memory Bandwidth:** The dual-socket architecture of the cluster provides higher throughput for the memory-intensive Mover operation.

6.4 Speedup Calculation

The speedup (S) achieved by the parallelization of the Mover operation is calculated using the formula:

$$S = \frac{T_{serial}}{T_{parallel}} \tag{1}$$

where T represents the average execution time of the Mover operation across 10 iterations. The results for the three problem indices on the Lab PC are summarized.

Table 1: Average Mover Execution Times and Calculated Speedup

Problem Index	**Avg. Serial (s)**	**Avg. Parallel (s)**	**Speedup (S)**
Problem 1	0.1741	0.1993	0.8735×
Problem 2	0.1717	0.2089	0.8219×
Problem 3	0.3444	0.1057	3.2583×

6.5 Discussion: Parallelizing Interpolation

If asked to parallelize the Interpolation function, the main challenge would be Race Conditions.

- **The Problem:** Multiple threads might try to update the same grid cell simultaneously if particles fall into the same or adjacent cells.
- **Approaches:** Using `#pragma omp atomic` for grid updates or creating thread-local grid copies (Reduction pattern).
- **Bottlenecks:** Atomic operations introduce high synchronization overhead, while thread-local grids consume excessive memory for large grid sizes (1000×400).